

Ζήτσημα 4.1 Να προγραμματίσετε σε **assembly** και να επιδείξετε στο Atmel Studio πρόγραμμα που να απεικονίζει ένα αναμμένο led που να αντιστοιχεί στα bit της **θύρας PB0-PB7**. Το αναμμένο led να κινείται συνεχώς ξεκινώντας από τα LSB προς τα MSB και αντίστροφα όταν φτάνει στο άλλο άκρο. Κάθε φορά το led να μένει αναμμένο για ~0.5 sec και το επόμενο led να ανάβει αμέσως. Η κίνηση του αναμμένου led να ελέγχεται από το **push button PA0**. Όταν αυτό είναι πατημένο η κίνηση να σταματάει, ενώ διαφορετικά να συνεχίζεται.

```
.include "m16def.inc"
```

```
.def reg=r16 ; ldi works on the upper 16 registers (r16 =r31)
```

```
reset:
```

```
ldi r24, low(RAMEND) ; initialize stack pointer
```

```
out SPL, r24
```

```
ldi r24, high(RAMEND)
```

```
out SPH, r24
```

```
ser r24 ; initialize PORTB for output
```

```
out DDRB, r24
```

```
clr r26 ; clear time counter
```

```
out DDRA, r26 ; initialize PORTA for input
```

```
a1:
```

```
ldi reg, 1 ; reg = 00000001
```

```
clc
```

```
rjmp moving_left
```

```
a3:
```

```
ldi reg, 2
```

```
clc
```

```
moving_left:
```

```
sbic PINA, 0 ; if (bit0 in PINA == 0), skip the next line
```

```
rjmp moving_left
```

```
out PORTB, reg
```

```
rcall loads
```

```
rol reg
```

```
brcs a2 ; if (carry == 1), go to a2
```

```
rjmp moving_left
```

```
a2:
```

```
ldi reg, 64 ; reg = 01000000
```

```
clc ; carry = 0
```

```
moving_right:
```

```
sbic PINA, 0 ; if (bit0 in PINA == 0), skip the next line
```

```
rjmp moving_right
```

```
out PORTB, reg
```

```
rcall loads
```

```
ror reg
```

```
brcs a3 ; if (carry == 1), go to a3
```

```
rjmp moving_right
```

```
loads:
```

```
ldi r24, low(500) ; load r25:r24 with 500
```

```
ldi r25, high(500) ; delay 0.5 second
```

```
rcall wait_msec
```

```
ret
```

wait_msec:

push r24 ; 2 ?????? (0.250 ?sec)

push r25 ; 2 ??????

ldi r24 , low(498) ; ?????? ??? ??????. r25:r24 ?? 998 (1
?????? - 0.125 ?sec)

ldi r25 , high(498) ; 1 ?????? (0.125 ?sec)

rcall wait_usec ; 3 ?????? (0.375 ?sec), ???????? ????????
???????????? 998.375 ?sec

; ??? ???? ?????? ??? ?????? ?????????????????!;

pop r25 ; 2 ?????? (0.250 ?sec)

pop r24 ; 2 ??????

sbiw r24 , 1 ; 2 ??????

brne wait_msec ; 1 ? 2 ?????? (0.125 ? 0.250 ?sec)

ret ; 4 ?????? (0.500 ?sec)

wait_usec:

sbiw r24 ,1 ; 2 ?????? (0.250 ?sec)

nop ; 1 ?????? (0.125 ?sec)

nop ; 1 ?????? (0.125 ?sec)

nop ; 1 ?????? (0.125 ?sec)

nop ; 1 ?????? (0.125 ?sec)

brne wait_usec ; 1 ? 2 ?????? (0.125 ? 0.250 ?sec)

ret ; 4 ?????? (0.500 ?sec)

Ζήτημα 4.2 Να υλοποιηθούν σε ένα σύστημα Μικροελεγκτή AVR (προσομοίωση στο Atmel Studio) οι λογικές συναρτήσεις:

$$F0 = ABC + B'C'D'$$

$$F1 = (A+B+C) \cdot D$$

Οι τιμές των $F0$ - $F1$ να εμφανιστούν αντίστοιχα στα δύο **LSB** της θύρας εξόδου **PORTA (0-1)**. Οι μεταβλητές εισόδου (A, B, C, D) υποθέτουμε ότι αντιστοιχούν στα 4 bit της θύρας εισόδου **PORTB (0-3)**, με το A στο LSB. Το πρόγραμμα να δοθεί σε **assembly** και σε **C**.

C

```
#include <avr/io.h>
```

```
char A, B, C, D, ROUT;
```

```
int main(void){
```

```
    DDRB=0x00;           // Αρχικοποίηση PORTB ως input
```

```
    DDRA=0xFF;           // Αρχικοποίηση PORTA ως output
```

```
    while(1){
```

```
        A = PINB & 0x01;    // Απομόνωση bit 0
```

```
        B = PINB & 0x02;    // Απομόνωση bit 1
```

```
        B = B >> 1;
```

```
        C = PINB & 0x04;    // Απομόνωση bit 2
```

```
        C = C >> 2;
```

```
        D = PINB & 0x08;    // Απομόνωση bit 3
```

```
        D = D >> 3;
```

```
        if (B != C)        ROUT = 0;
```

```
        else
```

```
        {
```

```
            if (B == 1){
```

```
                if(A == 1)    ROUT = 1;
```

```
                else ROUT = 0;
```

```
            }
```

```
            else             // Εδώ B = C = 0
```

```
            {
```

```

        if (D == 0)      ROUT = 1;

        else ROUT = 0;

    }

}

if (D == 1)    {

    if (A == 1 || B == 1 || C == 1)      ROUT = ROUT + 2;

}

PORTA = ROUT;

}

}

```

ASSEMBLY

```

.include "m16def.inc"

.def A = r16
.def B = r17
.def C = r18
.def D = r19
.def rout = r20

reset:

    ldi r24, low(RAMEND)  ; initialize stack pointer
    out SPL, r24

    ldi r24, high(RAMEND)
    out SPH, r24

    ser r24                ; initialize PORTA for output
    out DDRA, r24

    clr r24                ; initialize PORTB for input

```

```
out DDRB, r24
```

main:

```
in A, PINB
```

```
andi A, 1 ; isolation of 1st bit
```

```
in B, PINB
```

```
andi B, 2 ; isolation of 2nd bit
```

```
lsr B ; bring bit to lsb
```

```
in C, PINB
```

```
andi C, 4 ; isolation of 3rd bit
```

```
lsr C
```

```
lsr C
```

```
in D, PINB
```

```
andi D, 8 ; isolation of 4th bit
```

```
lsr D
```

```
lsr D
```

```
lsr D
```

F0:

```
cpse B, C ; if B != C then F0 = 0 and next line will not be skipped
```

```
rjmp F0_0 ; jump to F0_0 since F0 = 0
```

```
sbrc B, 0 ; if B = C = 0 we only care about B'C'D  
; since ABC = 0
```

```
rjmp ABC
```

BCD:

```
sbrc D, 0                ; if D = 0, skip next line
rjmp F0_0                ; if D = 1, F0 = 0
rjmp F0_1                ; if D = 0, F0 = 1
```

ABC:

```
sbrc A, 0                ; if A = 0, skip next line
rjmp F0_1                ; if A = 1, F0 = 1
rjmp F0_0                ; if A = 0, F0 = 0
```

F0_0:

```
ldi rout, 0
rjmp F1
```

F0_1:

```
ldi rout, 1
```

F1:

```
sbrs D, 0                ; if D = 1, skip next line
rjmp F1_0
```

```
sbrc A, 0                ; if A = 0, do not jump to F1_1
rjmp F1_1
```

```
sbrc B, 0                ; if B = 0, do not jump to F1_1
rjmp F1_1
```

```
sbrc C, 0                ; if A = 0, do not jump to F1_1
rjmp F1_1
```

F1_0:

```
out PORTA, rout          ; since F1 = 0, no need to change anything
rjmp END
```

F1_1:

```
ori rout, 2              ; rout <-- rout + 2
out PORTA, rout
```

END:

```
Nop
```

Ζήτημα 4.3 Να γραφτεί πρόγραμμα σε **C** για τον προγραμματισμό AVR Atmega16 και να γίνει προσομοίωση στο Atmel Studio 7 το οποίο αρχικά να ανάβει το led0 που είναι συνδεδεμένο στο bit0 της θύρας εξόδου **PORTB**. Στην συνέχεια με το πάτημα των διακοπών (Push-buttons) SW0-3 που υποθέτουμε ότι είναι συνδεδεμένα στα αντίστοιχα bit της θύρας εισόδου **PORTA** να συμβαίνουν τα εξής:

- SW0 ολίσθηση-περιστροφή του led κατά μια θέση **αριστερά** (κυκλικά από το LSB προς το MSB).
- SW1 ολίσθηση-περιστροφή του led κατά μια θέση **δεξιά** (κυκλικά από το MSB προς το LSB).
- SW2 μετακίνηση του αναμμένου led κατά δυο (2) θέσεις **αριστερά** (κυκλικά από το LSB προς το MSB).
- SW3 μετακίνηση του αναμμένου led στην θέση **MSB** (led7).

Υποθέτουμε ότι η αρχική του θέση του αναμμένου led είναι η **LSB** (led0). Όλες οι αλλαγές να γίνονται αφήνοντας (επανερχόμενα) τα Push-buttons SWx (bitx **PORTA**). Υποθέτουμε ότι τα push-buttons δεν θα πατιούνται ταυτόχρονα.

```
#include <avr/io.h>
```

```
char x;
```

```
int main(void)
```

```
{
    DDRB=0xFF; // Αρχικοποίηση PORTB ως output
    DDRA=0x00; // Αρχικοποίηση PORTA ως input
    x = 1;
    PORTB = x;
```

```

while (1)
{
    if ((PINA & 0x01) == 1){                // Έλεγχος πατήματος push-button SW0
        while ((PINA & 0x01) == 1);          // Έλεγχος επαναφοράς pushbutton
        if (x == 128)    x = 1;              // Έλεγχος υπερχείλισης
        else x = x << 1;                      // Αριστερή ολίσθηση κατά 1
    }
    else if ((PINA & 0x02) == 2){            // Έλεγχος πατήματος push-button SW1
        while ((PINA & 0x02) == 2);          // Έλεγχος επαναφοράς pushbutton
        if (x == 1)      x = 128;           // Έλεγχος υπερχείλισης
        else x = x >> 1;                      // Δεξιά ολίσθηση κατά 1
    }
    else if ((PINA & 0x04) == 4){            // Έλεγχος πατήματος push-button SW2
        while ((PINA & 0x04) == 4);          // Έλεγχος επαναφοράς pushbutton
        if (x == 128)    x = 2;              // Έλεγχος υπερχείλισης
        else if (x == 64) x = 1;             // Έλεγχος υπερχείλισης
        else x = x << 2;                      // Αριστερή ολίσθηση κατά 2
    }
    else if ((PINA & 0x08) == 8){            // Έλεγχος πατήματος push-button SW3
        while ((PINA & 0x08) == 8);          // Έλεγχος επαναφοράς pushbutton
        x = 128;
    }
    PORTB = x;
}

```

}

}